



GREEDY 알고리즘 스터디, 숲속의 기린

다이나믹 프로그래밍 (2)



by 요술토끼(curious.wzrabbit@gmail.com)

소개 및 학습법 ✨

- ✓ 다양한 유형과 아이디어를 사용하는 다이나믹 프로그래밍 응용 문제들을 해결해 봅시다!
- ✓ 풀이를 보시기 전 각 문제에 대해 충분한 고민을 해 보시는 것을 추천합니다.
 1. 기존 문제를 브루트포스 알고리즘과 같은 방법을 사용했을 때 효율성이 충분한가요?
충분하다면 다이나믹 프로그래밍을 굳이 도입할 필요가 없습니다.
만약 그리디 알고리즘을 알고 계시다면, 이 문제가 그리디한 전략이 통할까요? 통할 만한다면, 충분히 그리디 알고리즘을 시도해 볼 가치가 있습니다. 발상이 통하는지가 불확실하거나 무조건 특정 방법을 택하는 게 합리적이지 않다는 것이 보이는 경우 그리디 알고리즘 외의 다른 방법을 시도해 보는 것입니다.
 2. 효율성이 충분치 않고, 다이나믹 프로그래밍이 떠오르신다면, 필요한 정보가 무엇인지를 토대로 큰 문제와 작은 문제를 어떻게 생각해 볼 수 있을지 고민해 보세요.
 3. 그 다음, dp 테이블의 정의와 점화식을 생각하는 거예요.
 4. 마지막으로 시간복잡도를 통해 다이나믹 프로그래밍을 사용하는 풀이가 문제를 풀기에 충분한 효율성을 지녔는지 확인해 보세요.

소개 및 학습법 ✨

- ✓ 대신, 여러분은 이제 막 여러 유형들을 입문하는 입장이니, 잘 떠오르지 않으실 수도 있어요. 고민은 하되 너무 긴 시간 동안 하지 않으셔도 괜찮아요. 사람마다 다르지만 20~30분 정도 고민했는데도, 전혀 감이 잡히지 않으신다면 풀이를 보아도 충분하다 생각해요.
- ✓ 시간을 매우 길게 잡지 않는 이유는 충분히 고민해 본 상태에서 본 풀이들이 여러분의 시야를 넓혀주는 데 큰 도움이 되기 때문입니다. 때로는 몇몇 문제들의 풀이를 보고 시야를 넓힌 이후 더 어려운 문제들을 공략하는 게 더 효율이 좋을 수도 있습니다.
- ✓ 반대로 바로 답을 보는 것을 추천하지 않는 이유는 답을 보더라도 필요성이 잘 안 느껴지고, 금방 까먹기 쉽기 때문입니다. 그리고, 본 유형 특성상 현실의 문제를 어떻게 가공하여 아이디어를 떠올리는 감과 실력이 중요한데, 이 실력은 여러분이 고민하는 시간을 가져야 대체로 오릅니다.

소개 및 학습법 ✨

- ✓ 다이나믹 프로그래밍의 개념을 모르신다면, **다이나믹 프로그래밍 (1)** 자료를 먼저 읽으셔야 합니다! 본 자료에서는 별도로 개념을 설명하지는 않을 것입니다.



3

A. 정수 a 를 k 로 만들기 ✨

- ✓ 두 연산 모두 가면 갈수록 값이 커지는 연산입니다. 그렇다면 목표로 하는 값 K 로 가는 것을 큰 문제로 본다면 K 보다 작은 특정 값으로 가는 것은 부분 문제이자 작은 문제로 볼 수 있을 것입니다. 산의 정상에 오르기 전 여러 베이스캠프들을 지나치는 것처럼 비유할 수 있죠.
- ✓ $dp[i]$ = 정수 A 를 정수 i 로 만들기 위해 필요한 최소 연산 횟수로 정의해 봅시다.

3

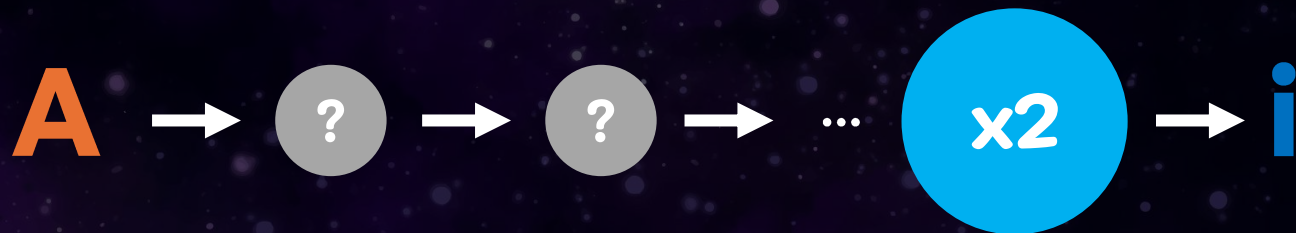
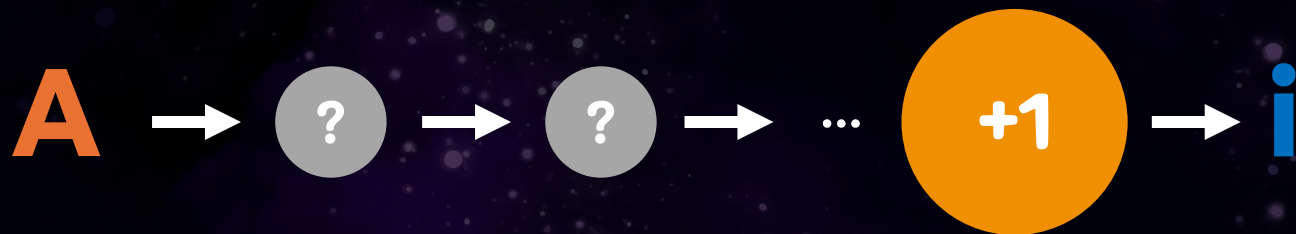
A. 정수 a 를 k 로 만들기 ✨

- ✓ 그렇다면, 어떻게 점화식을 세워야 할까요?
- ✓ 사실 이전 주치의 달나라 토끼를 위한 구매대금 지불 도우미 문제와 비슷한 아이디어입니다. “마지막이 xx 인 경우”를 기준으로 상황을 나누어 보세요.
- ✓ 더 고민해 보시고, 다음 페이지로 넘겨주세요.

3

A. 정수 a 를 k 로 만들기 ✨

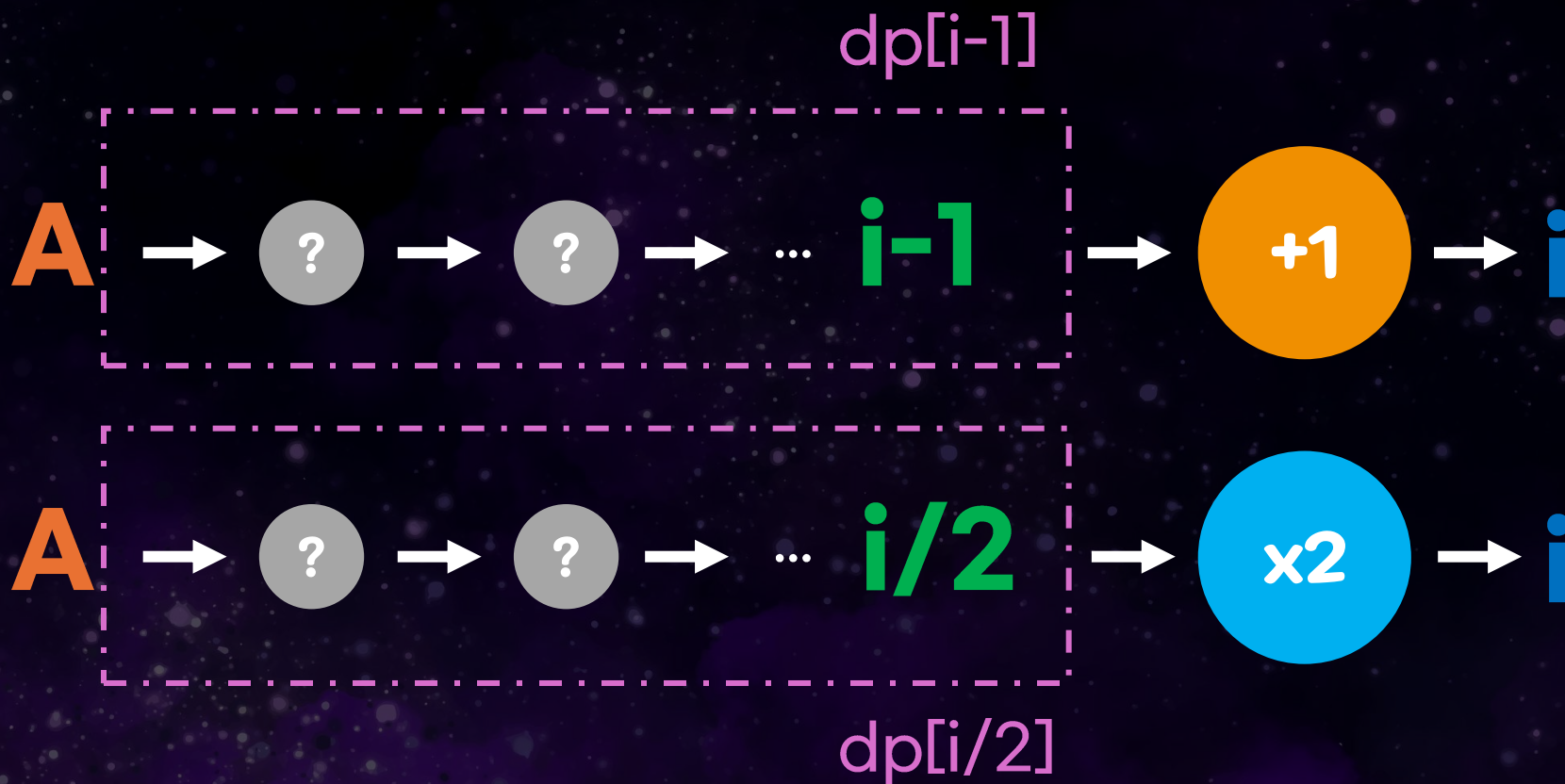
- ✓ 핵심은 바로 어떤 정수 i 로 만드는 것이 가능하다면, 그 중 가장 최적인 방법은 직전 수에 1을 더하는 방법과 2를 곱하는 방법 중 반드시 하나라는 것입니다.



3

A. 정수 a 를 k 로 만들기 ✨

- ✓ 그러면 문제가 이렇게 바로잡히고, 이미 구해둔 dp 테이블의 값을 활용하실 수 있다는 것을 알아낼 수 있습니다.



3

A. 정수 a 를 k 로 만들기 ✨

- ✓ 따라서 점화식은 아래와 같이 만들 수 있습니다. 왜 i 의 나머지에 따라 조건을 이렇게 나누는지도 고민해 보시면 좋겠습니다.

$$dp[i] = \begin{cases} \min(dp[i-1] + 1, dp[i/2] + 1) & \text{if } i \equiv 0 \pmod{2} \\ dp[i-1] + 1 & \text{if } i \equiv 1 \pmod{2} \end{cases}$$

*mod는 나머지를 의미합니다. 예를 들어 mod 2는 2로 나눈 나머지만입니다.

3

A. 정수 a 를 k 로 만들기 ✨

- ✓ 마지막으로, 초기값을 채울 때 주의할 점이 있습니다. 본 문제는 A에서 K로 만드는 초기 횟수를 구하는 것이므로, $A - 1$ 이하의 값으로는 도달이 불가능함에 유의해 주시고, 따로 구분 가능한 값을 채워 주셔야 값이 올바르게 업데이트 됩니다.
- ✓ 예를 들어 $A = 4, K = 6$ 인 경우 두 번 더하는 방법만 가능하므로 답은 2인데, $dp[3]$ 에 불가능함을 의미하는 값을 따로 채워주지 않으면 $dp[6] = dp[3] \times 2 + 1$ 와 같이 값을 계산하는 것이 가능해 답이 1로 잘못 나오는 상황이 벌어질 수도 있습니다.
- ✓ 개인적으로는 이 값을 ∞ 처럼 생각하면 좋을 것이라 생각합니다. 점화식을 사용할 때 이전 값이 ∞ 를 포함한다면 수가 매우 커지므로 자동으로 이 답이 채택되지 않거든요. $\infty = 1,000,000$ 이상이면 충분할 것입니다. 물론 -1과 같은 값으로 잡고 따로 값이 -1인 경우를 조건 분기하는 것도 유효합니다.

3

A. 정수 a 를 k 로 만들기 ✨

- ✓ ∞ 값을 고른다면 $\infty = 1,000,000$ 면 충분합니다. 가장 비효율적인 상황이 $A = 1, K = 1,000,000$ 인 상태에서 1만 더해서 도달하는 방법인데, 이 경우에도 999,999회만에 도달합니다. 따라서 답이 1,000,000 이상일 일은 없으므로 1,000,000 이상의 수를 ∞ 값으로 택하면 반드시 걸러질 것입니다.
- ✓ 시간복잡도는 $O(N)$ 입니다.



3

B. 1로 만들기

- ✓ 이전 문제에서는 어떤 연산을 쓰든 값이 증가했는데, 이번에는 어떤 연산을 쓰든 값이 감소합니다. 결국 생각만 반대로 해 주면 이전 문제와 비슷한 상황이 되고, 식만 다른 똑같은 발상을 쓰는 문제가 됩니다.
- ✓ 결국, 아래의 문제를 푸는 문제입니다!

문제

정수 X 에 사용할 수 있는 연산은 다음과 같이 세 가지 이다.

1. X 에 3을 곱한다.
2. X 에 2를 곱한다.
3. 1을 더한다.

정수 N 이 주어졌을 때, 위와 같은 연산 세 개를 적절히 사용해서 1을 N 으로 만들려고 한다. 연산을 사용하는 횟수의 최솟값을 출력하시오.

3

B. 1로 만들기

- ✓ $dp[i]$ = i 에서 시작해 1로 만들기 위해 필요한 연산의 수로 정의가 가능하며, 점화식은 아래 이미지로 올려두겠습니다.
- ✓ 식이 이전보다는 복잡하므로 특정 경우를 놓치는 일이 없도록 주의하세요. 예를 들어 6으로 나누어 떨어지는 경우 세 가지 연산을 전부 사용할 수 있는데, 3으로 나누는 경우가 항상 2로 나누는 경우보다 합리적이라 생각하고 2로 나누는 경우를 고려를 안 해 틀리는 경우가 있습니다. 잘못된 그리디 알고리즘 발상을 사용하는 경우입니다.
- ✓ 이번에도 역시 시간복잡도는 $O(N)$ 입니다.

$$dp[i] = \begin{cases} \min(dp[i/3] + 1, dp[i/2] + 1, dp[i - 1] + 1) & \text{if } i \equiv 0 \pmod{6} \\ \min(dp[i/3] + 1, dp[i - 1] + 1) & \text{if } i \equiv 0 \pmod{3} \text{ and } i \not\equiv 0 \pmod{2} \\ \min(dp[i/2] + 1, dp[i - 1] + 1) & \text{if } i \equiv 0 \pmod{2} \text{ and } i \not\equiv 0 \pmod{3} \\ dp[i - 1] + 1 & \text{otherwise} \end{cases}$$



B. 1로 만들기



✓ 보너스: 2, 3 둘 모두로 나눌 수 있는 경우 3으로 나누는 것이 더 이득인 경우: [링크](#)

2)가 3)보다 최적인 예로 642를 들 수 있습니다.

2)가 3)보다 최적임을 증명하기 위해, 처음에 642를 1로 바꾸기 위해 첫 번째 연산을 할 때만 각각 2, 3으로 나누고, 그 다음 연산부터는 3가지의 연산을 모두 고려한 최적값을 구하는 연산대로 해 봅시다.

입력: 642

1. 처음에 2로 무조건 나누고, 그 다음부터는 최적의 방법대로

642 --무조건 2로 나누기--> 321 -> 320 -> 160 -> 80 -> 40 -> 20 -> 10 -> 9 -> 3 -> 1

총 10번의 연산 필요

2. 처음에 3으로 무조건 나누고, 그 다음부터는 최적의 방법대로

642 --무조건 3으로 나누기--> 214 -> 107 -> 106 -> 53 -> 52 -> 26 -> 13 -> 12 -> 4 -> 3 -> 1

총 11번의 연산 필요

따라서, 경우에 따라서는 2와 3으로 모두 떨어지는 수라도 2로 나누는 게 더 최적일 수도 있습니다. 그러므로 항상 3가지의 연산을 모두 고려해 줘야 정답을 받는다고 할 수 있겠습니다!

3

C. $2 \times n$ 타일링 ✨

- ✓ 백준 질문 게시판에서 자세히 답변한 적이 있어, 링크를 첨부합니다. [여기](#)를 클릭해 이동해주세요.
- ✓ 질문글이 삭제될 것을 대비해 풀이 이미지들은 이곳에도 첨부해 두겠습니다. [여기](#)를 클릭해 바로 C번 문제 풀이의 마지막 페이지로 건너뛰세요.
- ✓ 본 문제 특성상 값을 몇 개 확인해보고 규칙으로 푸실 수도 있지만, 그래도 이 풀이를 확인해 왜 그런 규칙이 성립하는지를 확인해 보시면 좋을 거라 생각합니다.

dp[i] = 2 x i 크기의 직사각형을 1x2, 2x1 타일로 채우는 방법의 수

dp 테이블은 위와 같이 정의하겠습니다.

엄청 당연해 보일 수 있지만, 이 정의를 **끝까지 믿고 활용**하는 것이 중요합니다



dp[1], dp[2]를 구하는 것은 어렵지 않습니다
직접 구해봅시다

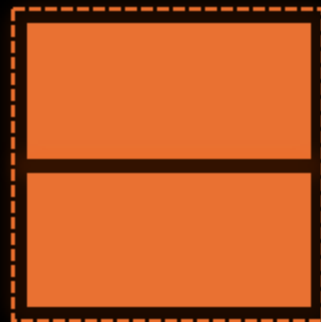
$$\text{dp}[1] = 1$$

2 x 1 크기의 직사각형을
1x2, 2x1 타일로 채우는 경우의 수



$$\text{dp}[2] = 2$$

2 x 2 크기의 직사각형을
1x2, 2x1 타일로 채우는 경우의 수



이 다음부터는, 직접 구하기보다는
이전에 구해 둔 정보를 활용해 봅시다

타일 전체를 칠하는 방법을 당장 알기는 어렵습니다.
하지만, 적어도 **가장 오른쪽의 타일을 채우는 방법은 아래의 두 가지 형태**뿐임은 알 수
있습니다

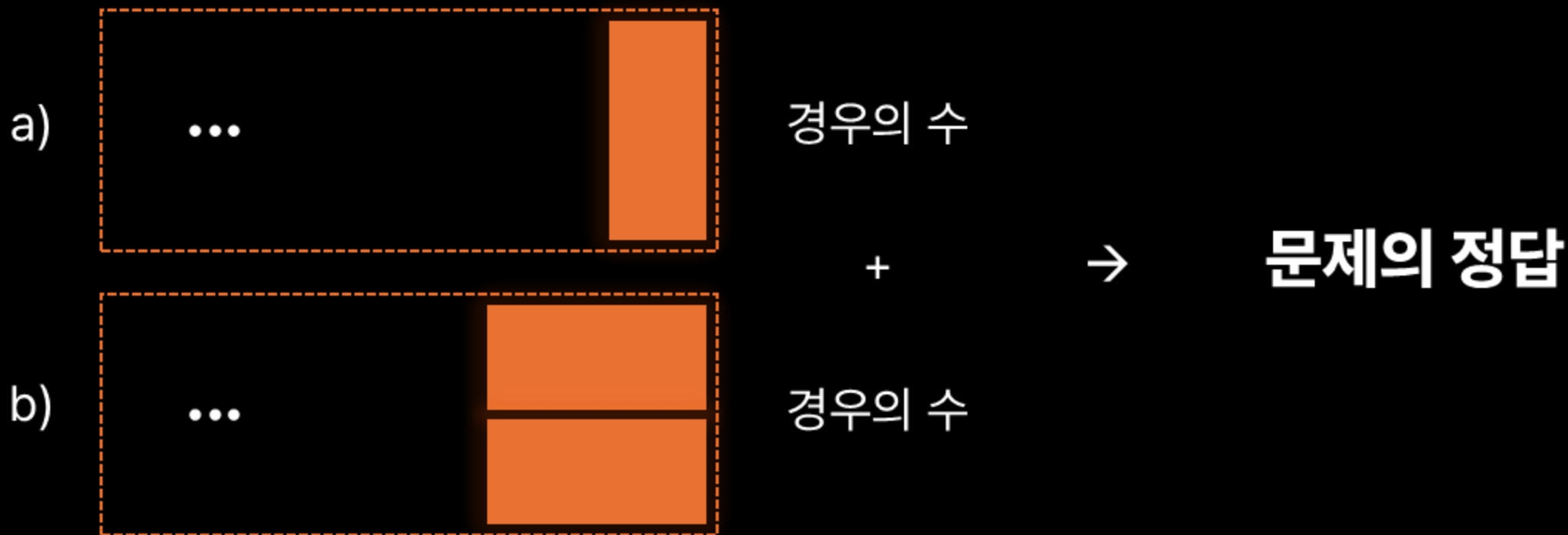


헛갈리기 쉬운 경우인데
이 경우는 독자적인 방법이 아니라,
마지막 타일이 2x1이므로 a) 에 속하는
경우입니다



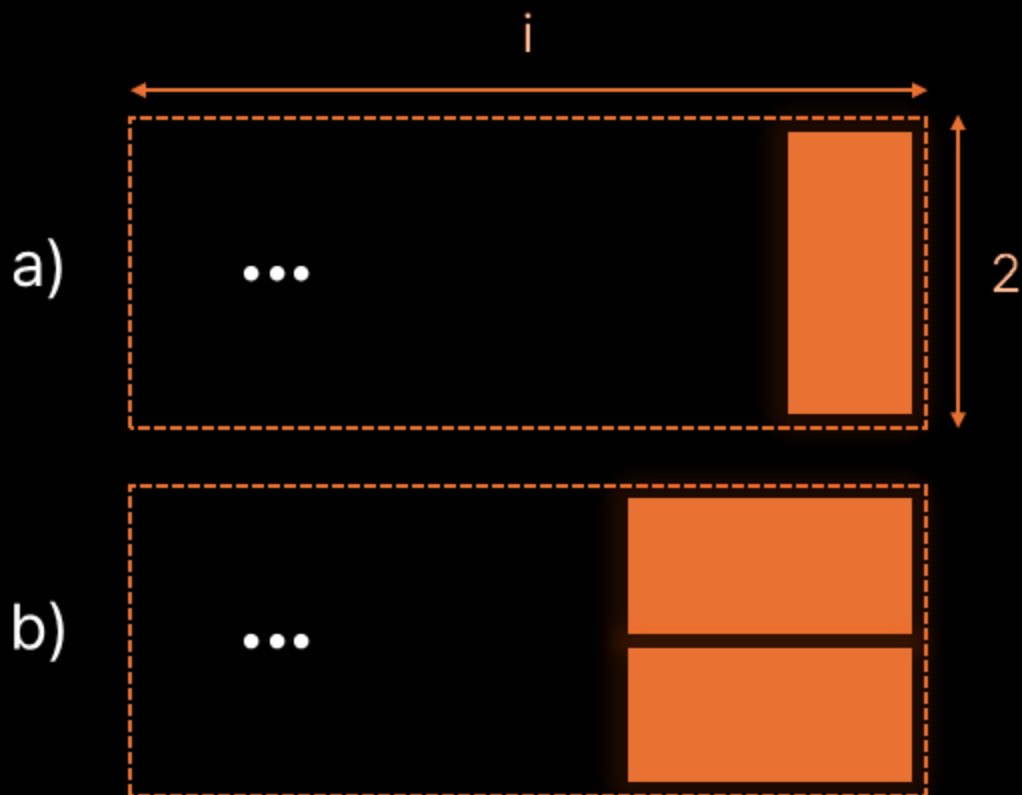
"a)" 방법으로 칠하는 경우와 "b)" 방법으로 칠하는 경우는 서로 겹치지 않습니다.
또한 두 방법 외에 마지막 타일을 칠하는 경우는 없습니다.

**따라서 "a)" 방법으로 칠하는 경우와 "b)" 방법으로 칠하는 경우의 수를 합하면 직사각형을
칠하는 전체 경우의 수와 같습니다.**



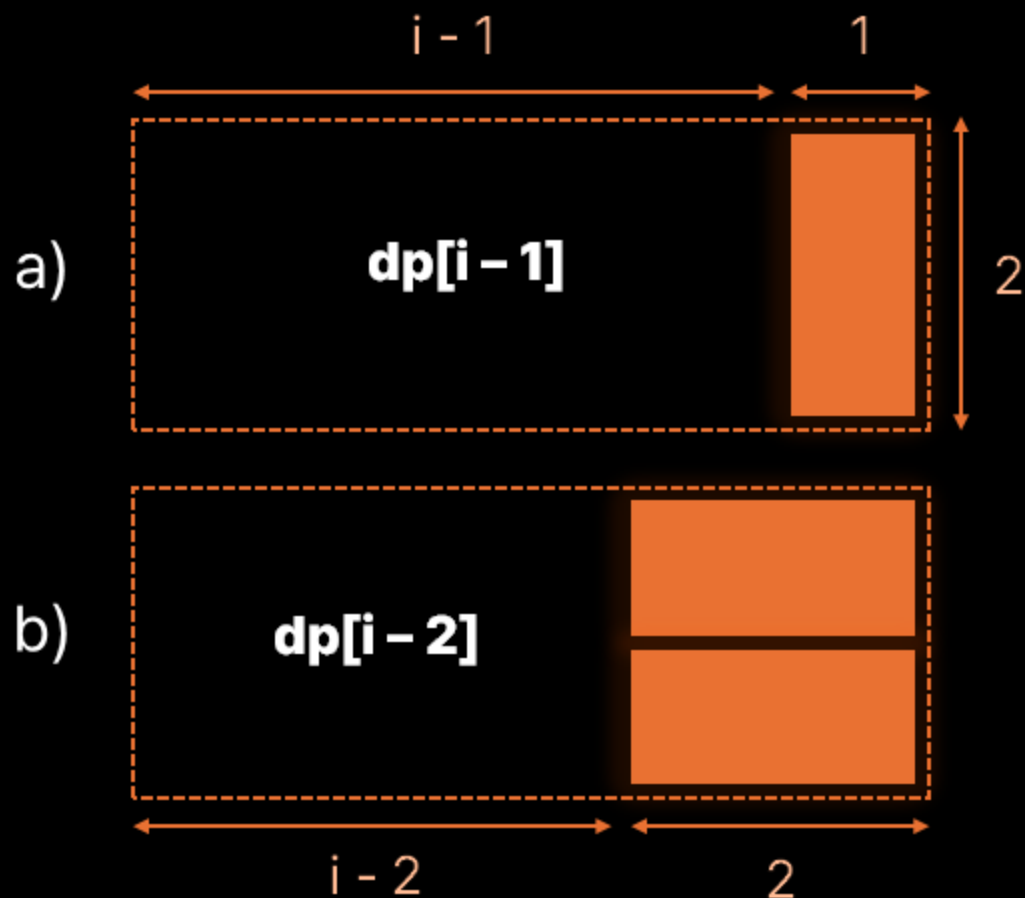
이제 이 원리를 활용해 이 상황을 점화식으로 만들어 보겠습니다.

2 x i 크기의 직사각형을 채웠을 때의 경우의 수를 구하려 합니다. 즉 **dp[i]**를 구하려 합니다.
마지막으로 칠하는 타일은 고정되어 있으므로, **나머지 공간을 타일로 채우는 경우의 수**를 생각해 보면 됩니다.

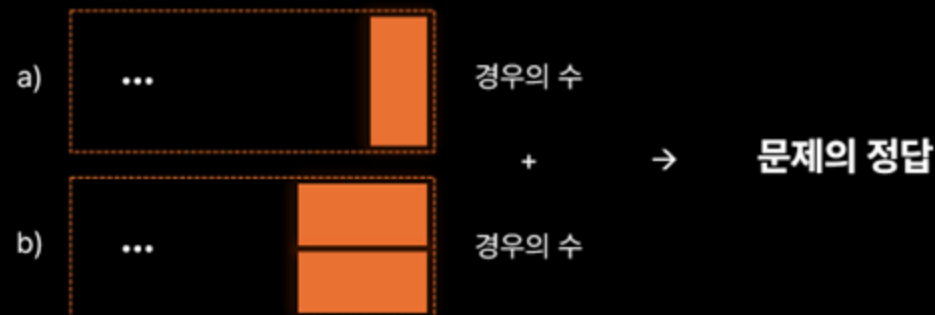


눈치채셨나요? **정의를 떠올려 활용**하면, 아래와 같이 나타낼 수 있게 됩니다.
큰 문제를 지금 작은 문제로 나누었습니다.

$dp[i] = 2 \times i$ 크기의 직사각형을 1×2 , 2×1 타일로 채우는 방법의 수



따라서 "a)" 방법대로 칠하는 경우와 "b)" 방법대로 칠하는 경우의 수를 합하면 직사각형을 칠하는 전체 경우의 수와 같습니다.



이제 이 사실을 그대로 활용하면...

이러한 형태의 점화식을 얻을 수 있게 됩니다.

a)

b)

$$\text{dp}[i] = \text{dp}[i - 1] + \text{dp}[i - 2]$$

그래서 피보나치 수열과 같은 형태로 답이 이어지는 것입니다.



처음에 $dp[1]$, $dp[2]$ 는 직접 구했습니다.
 $dp[3]$, $dp[4]$, ... 와 같은 더 큰 문제는 이전에 계산한 더 이전
단계의 dp 값을 이용한다면 답을 구해나갈 수 있습니다.

$$\begin{matrix} 3 & 2 & 1 \\ dp[3] = dp[2] + dp[1] \end{matrix}$$

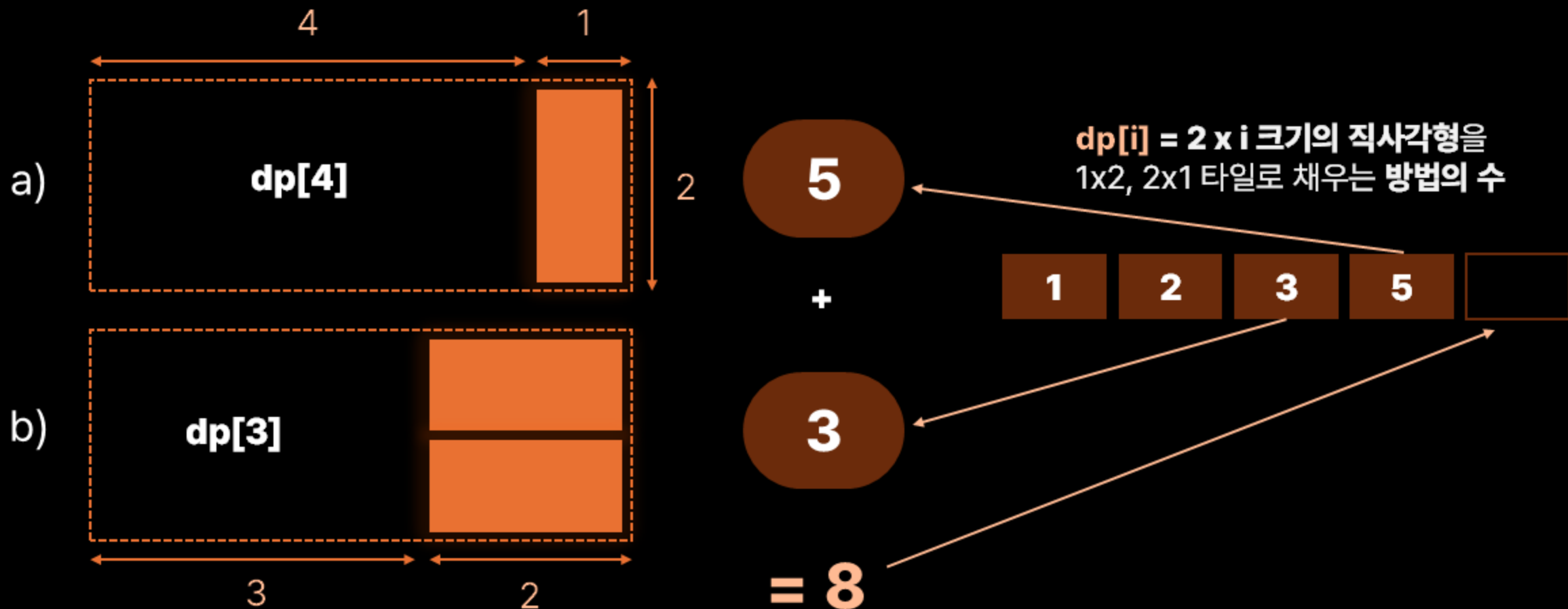
$$\begin{matrix} 5 & 3 & 2 \\ dp[4] = dp[3] + dp[2] \end{matrix}$$

$$\begin{matrix} 8 & 5 & 3 \\ dp[5] = dp[4] + dp[3] \end{matrix}$$

...



마지막으로, $dp[5]$ 를 구하는 과정을 그림으로 나타낸 모습입니다.



여기까지입니다, 행운을 빕니다!





D. 1, 2, 3 더하기 3 ✨

- ✓ 이번에는 경우의 수 구하기 문제입니다. 지금까지는 최적값을 구하는 문제였는데, 경우의 수 역시 구하는 것이 가능할까요?
- ✓ 물론입니다. 최적값을 구할 때에는 $dp[i] = \min(dp[?], dp[?], \dots)$ 와 같이 여러 경우들 중 하나를 골라 값을 선정했다면, 이번에는 $dp[i] = dp[?] + dp[?] + \dots$ 와 같이 여러 경우들을 모두 더한 값을 선정하면 됩니다.
- ✓ 이를 이용해 접근해 봅시다.

2 D. 1, 2, 3 더하기 3 ✨

- ✓ $dp[i]$ = i 를 1, 2, 3의 합으로 나타내는 경우의 수라고 정의해 보겠습니다.
- ✓ 특정 수를 1, 2, 3의 합으로 나타내는 경우는 1을 마지막 수로 고르는 경우, 2를 마지막 수로 고르는 경우, 그리고 3을 마지막 수로 고르는 경우의 3가지입니다.
- ✓ 그리고 각각의 경우에 대해 나머지 수를 고르는 경우의 수는 $dp[i - 1]$, $dp[i - 2]$, $dp[i - 3]$ 이 됩니다. 이 세 종류의 방법만이 i 를 1, 2, 3의 합으로 나타내는 경우이므로 세 가지 경우를 모두 더한 것이 답이 됩니다.



D. 1, 2, 3 더하기 3 ✨

- ✓ 답을 1,000,000,009로 나눈 값의 나머지가 답이므로 dp 테이블에 값을 채울 때마다 1,000,000,009로 나눈 나머지를 저장합니다.
- ✓ 모듈러 산술 연산은 분배 법칙이 성립합니다. 즉 아래 두 식의 결과는 같습니다. 그렇기 때문에 dp 테이블에 값을 채울 때마다 나머지 연산을 수행하더라도 답이 같습니다.
 1. $A \times B \times C \% \text{mod}$
 2. $A \% \text{mod} \times B \% \text{mod} \times C \% \text{mod}$
- ✓ 답을 한꺼번에 계산한 후 마지막에만 1,000,000,009로 나눈 나머지를 출력하는 방법으로 하지 않는 이유는 두 가지가 있습니다.
 1. 답이 기하급수적으로 커져 오버플로우가 일어날 수 있습니다.
 2. 오버플로우가 일어나지 않더라도, 수가 너무 커져 연산 속도가 급격하게 느려집니다.

2 D. 1, 2, 3 더하기 3 ✨

- ✓ 따라서, 앞선 정보들을 바탕으로 점화식은 아래와 같이 세울 수 있습니다.

$$dp[i] = (dp[i - 1] + dp[i - 2] + dp[i - 3]) \bmod 1\,000\,000\,009$$

- ✓ 시간복잡도는 $O(N)$ 입니다.

2 E. 연속합 ✨

- ✓ $dp[i]$ = i 번째 수가 마지막 수인 수열 중 합의 최댓값으로 정의해 봅시다. 또한 편의상 $arr[i]$ 를 수열의 i 번째 원소의 값이라고 해 봅시다.
- ✓ 우선, i 번째 수가 마지막 수인 수열 중 하나는 바로 i 번째 수만 있는 크기 1짜리 수열입니다. 이 값은 $arr[i]$ 입니다.
- ✓ i 번째 수가 마지막 수인 수열 중에서는 크기가 1보다 큰 수열도 있을 것입니다. 이 경우에는 $i - 1$ 번째 수를 반드시 수열의 원소로 택할 것입니다. 이 때 dp 테이블의 정의를 떠올린다면 $dp[i - 1]$ 은 곧 $i - 1$ 번째 수가 마지막 수인 수열 중 합의 최댓값임을 알 수 있으며, 따라서 $dp[i - 1] + arr[i]$ 가 $i - 1$ 번째 수를 반드시 수열의 원소로 택했을 때의 최댓값임을 알 수 있습니다.
- ✓ i 번째 수가 마지막 수인 수열은 $i - 1$ 번째 수를 수열의 원소로 택하는 경우와 택하지 않는 경우 두 가지 뿐이니, 이 두 방법을 통해 우리가 원하는 값을 구할 수 있습니다.

2 E. 연속합 ✨

- ✓ i 번째 수가 마지막 수인 수열은 $i - 1$ 번째 수를 수열의 원소로 택하는 경우와 택하지 않는 경우 두 가지 뿐이니, 이 두 방법을 통해 우리가 원하는 값을 구할 수 있습니다.
- ✓ 따라서, 점화식은 아래와 같이 정리할 수 있습니다.

$$dp[i] = \max(arr[i], dp[i - 1] + arr[i])$$

$i - 1$ 번째 수를 수열의 원소로
택하지 않는 경우

$i - 1$ 번째 수를 수열의 원소로 택하는 경우

2 E. 연속합 ✨

- ✓ 이번에는 N번째 원소가 수열의 마지막 원소인 경우의 합이 최대라는 것이 보장되지 않습니다. 1번째 원소가 마지막 원소인 경우, 2번째 원소가 마지막 원소인 경우, ... 가 모두 답의 후보가 될 수 있습니다. 따라서 $dp[N]$ 이 답이라는 보장이 없습니다. 여러분은 $\max(dp[1], dp[2], \dots, dp[N])$ 을 구해야 합니다.
- ✓ 또한, 연속합은 반드시 1개 이상의 수를 골라야 함에 유의해야 합니다. 따라서 답을 저장하는 변수의 값을 0으로 초기화한 후 갱신하는 경우 틀릴 수 있습니다.
- ✓ 시간복잡도는 $O(N)$ 입니다.

2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ LIS(Longest Increasing Subsequence)라 불리는 유명한 유형의 문제입니다. 이 발상을 베이스로 하는 문제가 많으므로 숙지해두시면 좋습니다.
- ✓ 용어가 길기에 편의상 가장 긴 증가하는 부분 수열을 LIS라 부르겠습니다. 자연스럽게 우리가 구해야 하는 가장 긴 증가하는 부분 수열의 길이는 LIS의 길이가 됩니다.

2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 이번에도, 연속합 문제와 비슷하게 마지막 원소에 초점을 두고 dp 테이블의 정의를 세워볼 수 있습니다. $dp[i]$ = i번째 원소가 마지막 원소인 LIS의 길이로 정의해 봅시다. $arr[i]$ 는 수열의 i번째 원소의 값입니다.
- ✓ 정의대로 했을 때, $dp[1] = 1$ 이 됩니다. 1번째 원소가 마지막 원소인 LIS는 반드시 자기 자신을 고른 경우 단 한 가지이며, 길이가 1이 되기 때문입니다.

2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 이미 오름차순으로 정렬되어 있는 두 수열을 잇고 싶습니다. 이는 이후에도 하나로 합쳐진 수열은 오름차순을 이루도록 유지시키고 싶습니다. 이 경우를 만족시키려면 조건은 어떻게 될까요?
- ✓ 바로, 좌측 수열의 마지막 원소보다 우측 수열의 첫 번째 원소의 값이 크기만 하면 됩니다. 다른 원소들의 값이 어떤지는 중요하지 않죠. 이 점을 핵심 발상으로 삼아, 수열들을 이어나갈 것입니다.

2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ dp 테이블은 아래와 같이 채워나갈 것입니다.
- ✓ i 번째 원소를 마지막 원소로 하는 수열은
 - ① 다른 아무 원소도 포함하지 않거나,
 - ② $1, 2, 3, \dots, i-1$ 번째 원소 중 하나를 마지막에서 두 번째 원소로 포함하는 경우 두 가지로 나눌 수 있습니다.
- ✓ ①의 경우 수열의 길이는 1입니다. 이러한 수열은 항상 존재하므로, LIS의 최솟값은 1입니다. 답이 0 이하로 떨어질 일은 없겠죠.

2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ ②의 경우 이전에 구했던 $dp[1], dp[2], \dots, dp[i - 1]$ 을 재활용하면 도움이 됩니다. $dp[i] = i$ 번째 원소가 마지막 원소인 LIS의 길이이니까요. 마지막에서 두 번째로 올 원소만 고르면 그 원소를 골랐을 때의 LIS의 길이는 반드시 dp 테이블의 값에 1을 더한 값이 될 것입니다.
- ✓ 단, 이번에는 항상 그럴 수는 없습니다. 마지막에서 두 번째 원소의 값이 $arr[i]$ 보다 크거나 같다면, 그 원소와 i 번째 원소를 이을 수 없기 때문입니다. 따라서 여러분은 이 부분을 별도로 확인해 주셔야 합니다.
- ✓ 즉 마지막에서 두 번째로 올 수 있는 원소들에 한하여 LIS가 될 수열을 연결시켰을 때의 LIS의 길이가 얼마인지를 계산하고, 이 중 최댓값이 되는 경우를 채택하는 식으로 dp 테이블을 채울 수 있습니다.
- ✓ 과정이 생소할 수 있으므로 예제 1번을 그림으로 다시 설명하겠습니다.

2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 초기에, dp 테이블은 비어 있는 상황입니다. 이전 원소랑 이어서 수열을 만들 수 있는지를 확인해야 하므로 원소의 값들은 모두 가지고 있겠습니다.

원소의 값

10

20

10

30

20

50

dp 테이블



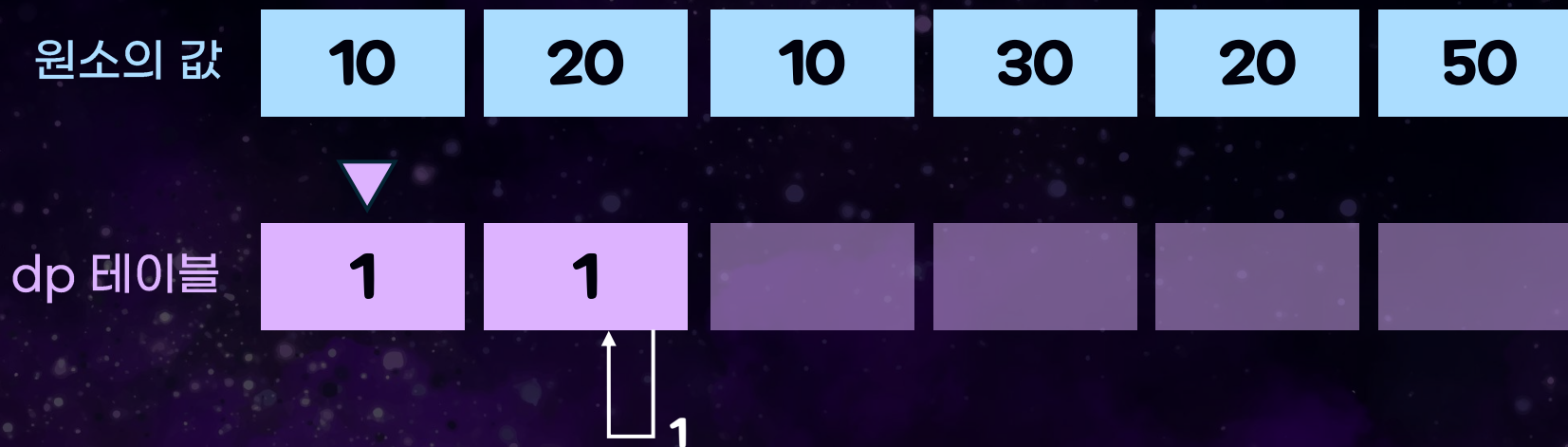
2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ $dp[1] = 1$ 을 채우겠습니다. 첫 번째 원소가 마지막 원소인 수열은 자기 자신 하나만을 고르는 경우가 유일하기 때문입니다.
- ✓ dp 테이블의 각 값은 앞서 다뤘듯이 1 이상일 것입니다. 이를 이용해 dp 테이블의 각 값을 1로 초기화시키는 것도 가능합니다. 하지만 헷갈리므로 여기에서는 그렇게 하지 않겠습니다.

원소의 값	10	20	10	30	20	50
dp 테이블	1					

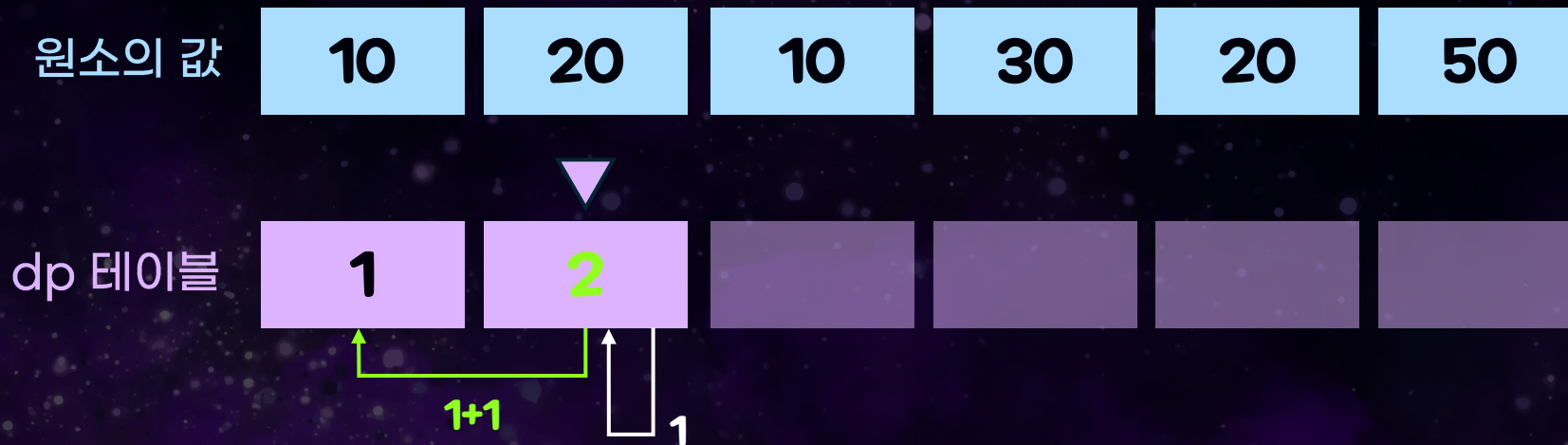
2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 이제 $dp[2]$ 를 채워 봅시다. 우선 두 번째 원소를 마지막 원소로 채택한다면, 선택지는 두 가지입니다. 두 번째 원소만을 수열로 하는 길이 1의 수열을 만들 것인가, 아니면 첫 번째 원소를 마지막에서 두 번째 원소로 채택할 것인가.
- ✓ 전자를 택하면 $dp[2] = 1$ 이 됩니다.



2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 첫 번째 원소를 마지막에서 두 번째 원소로 택하는 것이 가능할까요? 이번에는 가능합니다. $arr[1] = 10, arr[2] = 20$ 으로 $arr[2] > arr[1]$ 이기 때문입니다.
- ✓ 따라서 자기 자신만을 원소로 택하는 경우와 첫 번째 원소를 마지막에서 두 번째로 택하는 경우 중 더 최적인 경우는 후자입니다.
이 경우 $dp[2] = dp[1] + 1 = 2$ 이 됩니다.
 j 번째 원소를 고르는 것이 가능하다면 점화식은 $dp[i] = \max(dp[i], dp[j] + 1)$ 이 됩니다.



2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 세 번째 원소도 같은 방법으로 해 봅시다. 자기 자신만을 수열로 택하는 것은 항상 가능하므로 우선 $dp[3] = 1$ 이 됩니다.

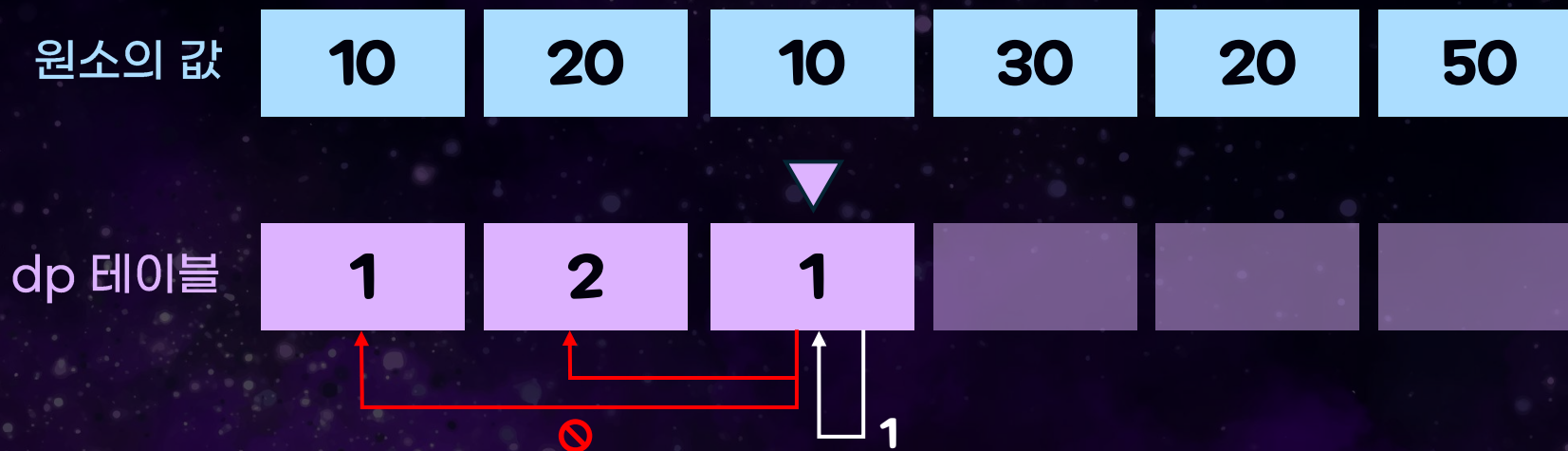
원소의 값	10	20	10	30	20	50
dp 테이블	1	2	1			

Diagram illustrating the calculation of the Longest Increasing Subsequence (LIS) for the third element (10) in the array [10, 20, 10, 30, 20, 50].

The diagram shows the original array and the corresponding DP table. The third element (10) is highlighted, and an arrow points to its value in the DP table, which is 1. This indicates that the longest increasing subsequence ending at the third element has a length of 1, as it can only include the element itself.

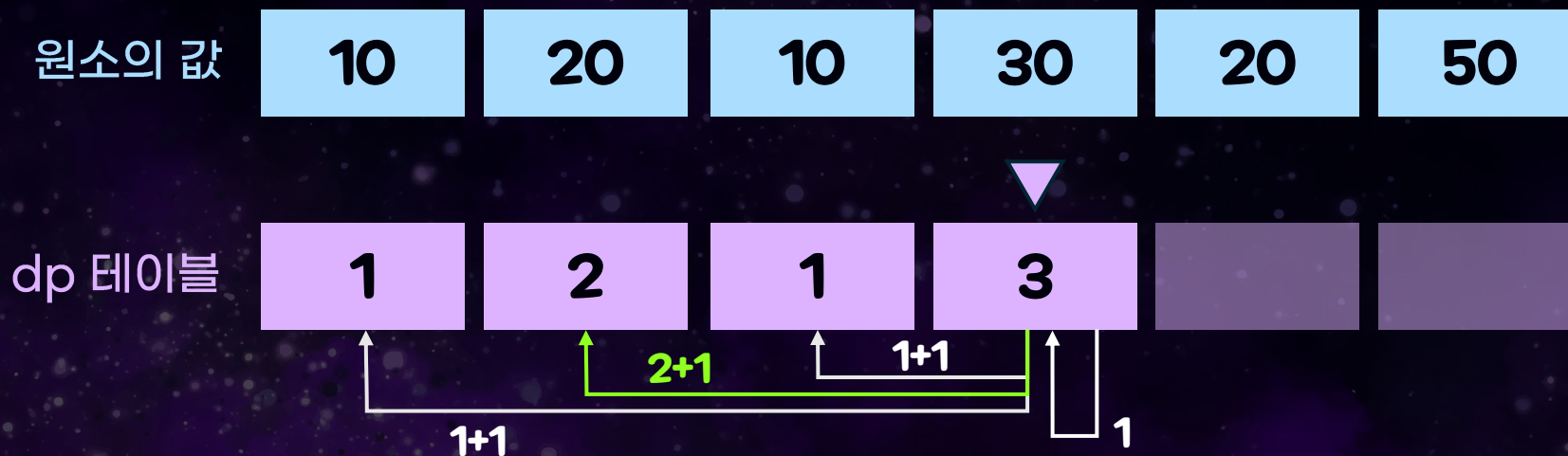
2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 이번에는 딱하게도 $arr[1]$, $arr[2]$ 모두가 $arr[3]$ 보다 작거나 같습니다. 그렇기 때문에 마지막에서 두 번째 원소로 택할 수 있는 원소가 없습니다.
- ✓ 따라서 $dp[3] = 1$ 로 확정됩니다.



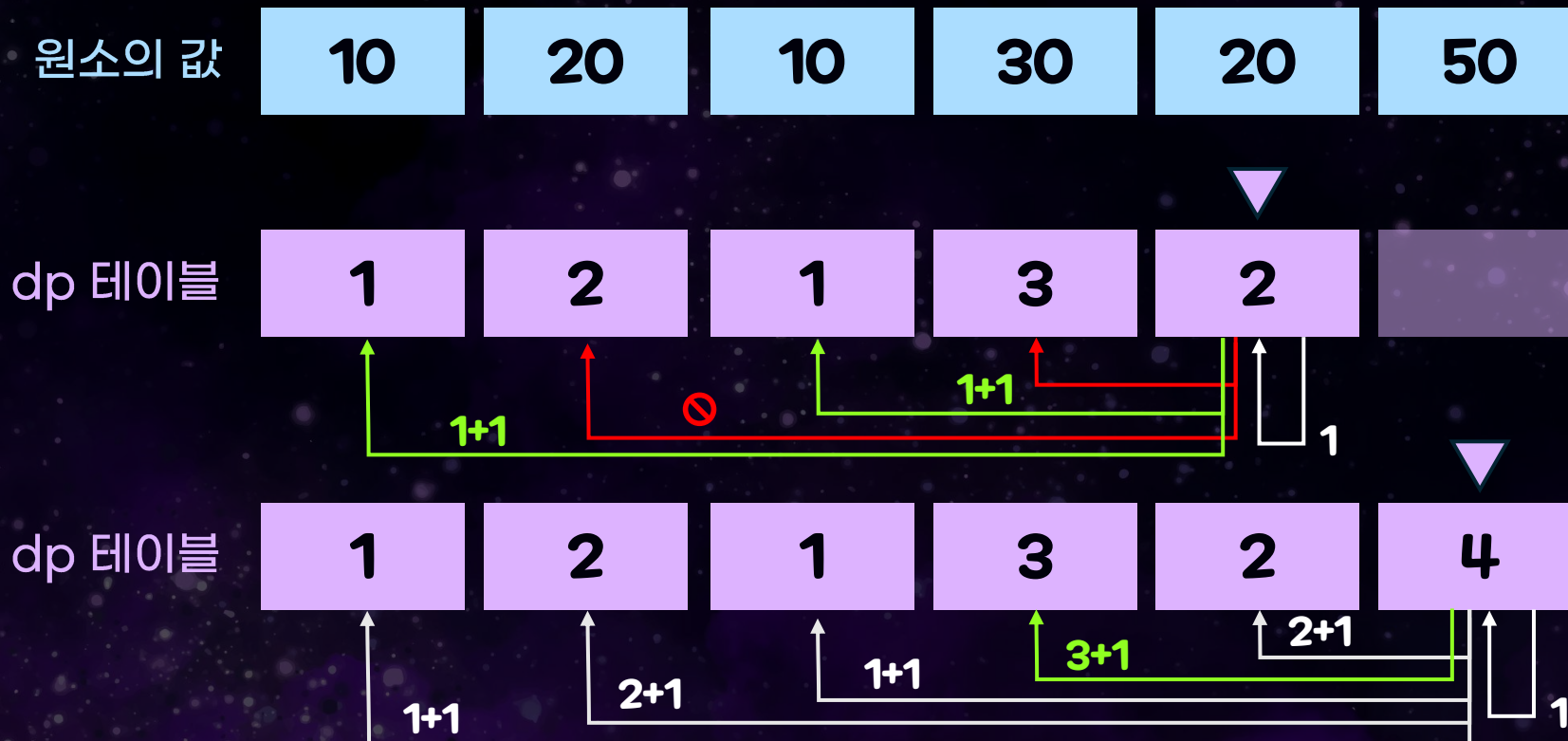
2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 네 번째 원소도 진행해 보죠.
- ✓ 이번에는 세 원소 모두 마지막에서 두 번째 원소로 택할 수 있는 상황이고, 그 중 두 번째 원소를 택하는 경우가 가장 최적인 상황입니다. $dp[4] = 3$ 이 됩니다. 두 번째 원소를 마지막 원소로 두는 수열의 LIS는 $[10, 20]$ 이고, 길이는 2이죠. 이 뒤에 $[30]$ 이라는 수열을 이은 것입니다.



2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ 다섯 번째, 여섯 번째 원소도 진행해 보죠. 이제는 dp 테이블이 왜 이렇게 결정되는지 이해가 되실 거라 믿습니다.



2 F. 가장 긴 증가하는 부분 수열 ✨

- ✓ E. 연속합 문제와 마찬가지로 이 문제 또한 마지막 원소가 N번째 원소인 LIS가 답이라는 보장이 없습니다. $\max(dp[1], dp[2], \dots, dp[N])$ 을 구하셔야 합니다.
- ✓ 시간복잡도는 $O(N^2)$ 이지만, 원소의 개수가 최대 1,000개이기 때문에 이 효율성으로도 충분히 문제를 맞출 수 있습니다.
- ✓ 보너스: 만약 LIS의 길이뿐만 아니라 LIS 그 자체를 구해야 한다면 그 때에는 어떻게 접근해 볼 수 있을까요? 문제



The End